

Citation Cleaner

From a topic, an author, or a paper title — to the references that actually matter, ranked by how often the field cites them, with every citing passage traced back to its source.

Who, when, and what for

Before the machinery, the use case. Citation Cleaner is built for the moment when you are staring at an unfamiliar research area and need to know **which works the literature keeps coming back to** — not just a flat list of references, but a frequency-ranked, evidence-backed map of the conversation.

Who it's for

- Researchers entering a new field
- Grad students writing a lit review
- R&D / policy landscape scans
- Librarians & research support

When to use it

- You have a topic, not a ref list
- You want the most-cited works first
- You need to know *where* it's cited
- You're cleaning refs at scale

What it does

- Find top-relevant papers
- Mine & rank shared citations
- Surface in-context passages
- Normalize & dedupe references

The headline feature: citation frequency, with receipts

Give the agent a starting signal — an **author**, a piece of **terminology**, or a **paper title** — and it retrieves the most relevant papers in the corpus, say the **top 10**. It then opens each of those papers and reads *who they cite*. Every reference is tallied across the set and ranked from most to least cited.

The payoff is a report that reads like a field guide. If Paper A appears in 9 of the 10 papers and Paper B in 8, the report leads with them — and for each, it shows **exactly where** the citation happened: the in-context passage, the page number, and the citing paper.

Example output

Paper A — cited **9×** across the top-10 set.

- in *Zhang et al., 2023*, p. 4: "...building on the estimator proposed by [A]..."
- in *Okoye, 2022*, p. 11: "...we adopt the same blocking strategy as [A]..."
- ...and 7 more citing passages, each linked to its page and source.

The challenge: why this task is genuinely hard

Ranking citations *sounds* like counting. It isn't — and that gap is the whole reason this is an agent rather than a script.

1. The same work wears a dozen disguises

A single paper shows up across a corpus as reordered author names, initials versus full names, drifting title casing, typos, missing venues, and edition variants. Count naively and you split one work into five. The system's first job is to collapse every messy variant into **one canonical entry per unique work** *before* it counts anything — otherwise the frequency ranking is meaningless.

2. Look-alike titles are not the same work

The mirror-image failure: two genuinely different books are both titled *Generalized Additive Models*. Merge them and you fabricate a “highly cited” work that never existed. Telling these apart often requires reading the *context* in which each was cited — exactly the kind of judgment a regex cannot make.

3. The hard cases aren't catchable by rules

The difficulty comes in layers, and only the top layer is tractable with deterministic tooling:

Layer	Example	Regex / fuzzy?
Format noise	extra spaces, page suffixes, & versus and	yes
Field order	Smith, J. versus J. Smith	partly
Abbreviation	John R. Smith versus J. R. Smith	no
Casing & typos	asymptotic statistics versus misspelled variants	no
Entity ambiguity	two different books titled <i>Generalized Additive Models</i>	no
Cross-corpus context	same raw reference used in different citing papers	no

4. The input itself is messy

Real corpora don't arrive as clean reference tables — they arrive as PDFs. Locating the reference section, splitting it into entries, and linking each in-text marker back to its source (so you can say “cited on page 4”) is brittle, template-dependent work that no single rule covers.

Why an agent + LLM, not a script

Put together, these are not formatting problems — they are **judgment problems**. Deciding whether two strings name the same work, or whether two identically titled books are distinct, requires reading meaning, not matching patterns. That is where the LLM earns its place. But model calls are expensive, so the architecture is deliberately frugal:

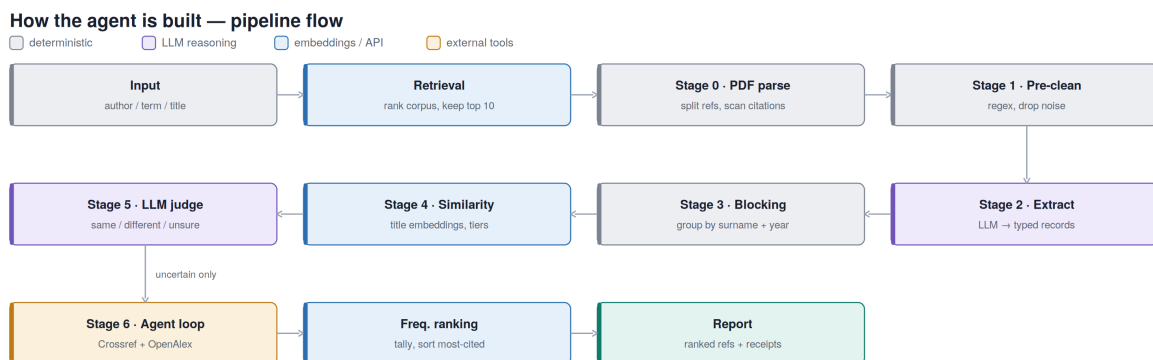
- **Deterministic stages do the bulk** of the work wherever they are trustworthy — cleanup, blocking, counting.
- **The LLM is spent only on the residue** — parsing irregular strings, and judging clusters where similarity is ambiguous.

- **The agent loop escalates further** only for clusters the judge still can't resolve, calling Crossref, OpenAlex, and citing-context lookups to break the tie.

The result is a pipeline that uses cheap tools for easy cases and reserves real reasoning for the cases that actually need it.

How the agent is built: pipeline architecture

Eight stages turn a query (or a folder of PDFs, or a raw reference CSV) into a ranked, traceable citation report. The flow snakes left-to-right, then back, then forward again. Color marks the kind of work each stage does — and shows at a glance how sparingly the LLM is used.



Flow: Input → Retrieval → Stage 0–6 → Frequency ranking → Report. The “uncertain only” branch shows that the agent loop runs solely on clusters the LLM judge could not settle.

Where the LLM is used (and where it isn't)

Only **Stage 0 fallback**, **Stage 2**, and **Stage 5** are the model's territory — irregular parsing and same/different judgment. Everything else is deterministic code or embedding/lookup. The model is the scalpel, not the hammer.

Two entry points, one pipeline

CSV mode starts at Stage 1 with a hand-curated reference table. **PDF mode** adds Stage 0, which produces the raw references *and* the in-text citing context that powers the headline report.

Stage 0: the PDF ingestion channel

Stage 0 is the part worth noticing. It makes the system usable when your starting point is a folder of papers rather than a tidy CSV — and it is where the in-context citing passages that power the headline report are born.

- 1. Extract page text from each PDF with PyMuPDF.
- 2. Locate the reference section by heading and position signals.
- 3. Split that section into individual raw reference strings.
- 4. Score extraction quality by reference count, year coverage, and author-like starts.
- 5. Optionally call an LLM fallback when reference-section extraction looks weak.
- 6. Extract header metadata: title, DOI, arXiv id, year.
- 7. Detect numbered and author-year in-text citation markers.
- 8. Link each marker back to its reference entry — the basis for “cited on page N”.
- 9. Write a ParsedDocument with references, citances, metadata, and quality signals.

Not a black box

Every PDF produces inspectable artifacts. A bad section cut or an unlinked citation is visible *before* it can poison the canonical reference table or skew the frequency ranking.

Boundary

The PDF channel is wired for publisher PDFs with embedded, selectable text. It is **not** an OCR system for scanned documents, and it does not promise full layout reconstruction for every journal template. Low-quality parses are surfaced through quality reports and review artifacts — never silently waved through as success.

What changed from the prototype

Area	Prototype design	Current implementation
Input surface	Raw reference strings supplied by the user	PDF files or raw reference CSVs. Stage 0 now writes <code>parsed_documents.jsonl</code> , <code>raw_refs.csv</code> , and <code>citances.jsonl</code> .
Architecture	Script-oriented staged pipeline	Importable typed package with schemas, a parser layer, a stage abstraction, and two orchestrators.
PDF reading	Out of scope	Connected through <code>HeuristicParser</code> : page-text extraction, reference-section split, metadata extraction, citation-marker scan, quality fallback.
Citance context	External optional input	Automatically produced in PDF mode; available to Stage 6 for disambiguation and to the report for in-context sourcing.
Schemas	Shared script module	Dedicated Pydantic schema layer under <code>citation_cleaner/schemas</code> .

Area	Prototype design	Current implementation
Resume behavior	Intermediate files by script	Stage artifacts load automatically from the workdir; --resume-from recomputes from a named stage.
Testing	Conceptual smoke path	Deterministic tests cover schemas, precleaning, clustering, parser splitting, citance linking, and dry-run execution.

Repository layout

```
package-root/
  SKILL.md
  citation_cleaner/
    schemas/      reference.py, citance.py, document.py
    parsers/      base.py, heuristic.py, reference_section.py,
                  citance_linker.py, llm_fallback.py, quality.py
    preclean/     rules.py
    extractors/   llm.py, dry_run.py, cache.py
    blocking/     surname.py
    embedding/    providers.py, clustering.py
    resolvers/    judge.py, agent.py, tools.py
    pipelines/    stages.py, pdf_to_canonical.py, refs_to_canonical.py
  scripts/
    run_pipeline.py  parse_pdfs.py  eval_stage0.py
    eval_score.py   check_api.py
  references/
    schema.md  pdf_parsing.md  prompts.md  tools.md  eval.md
  tests/
    test_smoke.py  test_parsers.py  test_citance_linker.py  test_pipelines.py
```

The package is self-contained: `SKILL.md` is the agent-facing contract, `citation_cleaner/` is the importable implementation, and `scripts/`, `references/`, and `tests/` are bundled resources.

How to use the skill

Use `citation-cleaner-v2/` as the skill folder. Read `SKILL.md` first, then reach for the bundled scripts only as the task demands. Commands run from the skill directory.

Raw reference-table workflow

```
python scripts/run_pipeline.py \
  --refs demo/raw_refs_sample.csv \
  --workdir run_refs --dry-run
```

PDF corpus workflow

```
python scripts/run_pipeline.py \
  --pdfs ./papers \
  --workdir run_pdfs --dry-run
```

Real API run, after credentials pass

```
python scripts/check_api.py
python scripts/run_pipeline.py \
  --pdfs ./papers --workdir run_real
```

Evaluation

Stages 1 through 6 are scored with pair-level F1 and cluster purity against a hand-labeled or curated reference set. Stage 0 earns its own PDF-level metrics: reference recall, reference precision, citance

recall, citance-link accuracy, fallback rate, and whether the fallback actually improved extraction quality.

The governing question

How much does PDF parsing degrade the downstream canonicalization — and therefore the citation ranking — compared with hand-extracted reference strings from the same papers? The production target is a **small downstream pair-F1 delta**, not just a nice-looking parser demo.